

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA1011	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	AKTCHAYA A	Reg. No.:	192424096
Date:	20/08/2026	Duration:	60 minutes

Code of Conduct

I AKTCHAYA A certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
 - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
- Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository	README, installation guide, usage instructions, and documentation are	Documentation is mostly complete.	Basic documentation	Poor or insufficient documentation.

Maintenance(15 %)	complete, clear, and well organized.		with missing details.	
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

Annexure A – Code Review & Repository Evaluation

1. Problem Overview

- **Problem Statement**

The **University Academic Management and Student Success Platform** is designed to provide a centralized digital system for managing academic activities within a university. Higher education institutions handle large volumes of student, faculty, course, attendance, examination, and academic performance data. Managing these activities through separate systems or manual processes can result in data duplication, delayed reporting, communication gaps, and difficulty in monitoring student progress.

The proposed platform integrates major academic activities into a single system. It manages student admissions, course registration, attendance, assessments, examination results, academic records, faculty activities, and learning progress.

The system also monitors student academic performance and attendance using configurable academic policies. Based on predefined criteria, it can identify students who are academically at risk and notify faculty members and academic advisors for timely intervention.

- **Implemented Solution**

The proposed solution is a centralized web-based academic management platform that provides different functionalities for students, faculty members, academic advisors, and administrators.

The platform provides:

- Student profile and academic record management.
- Course registration and enrollment management.
- Attendance monitoring.
- Assignment and assessment management.
- Examination and result management.
- Faculty activity management.
- Student performance monitoring.
- At-risk student identification.
- Notifications and alerts.
- Academic reports and dashboards.
- Administrative monitoring.
- Centralized academic information management.

- **Project Objectives**

The main objectives of the system are:

- To manage student academic records in a centralized platform.
- To monitor student attendance and academic performance.
- To provide an efficient course registration system.
- To manage examinations and academic assessments.
- To notify students, faculty members, and advisors about important academic activities.
- To generate academic and institutional reports.
- To identify academically at-risk students at an early stage.
- To improve institutional academic planning.
- To enhance student engagement and communication.
- To support data-driven academic decision-making.
- To reduce manual administrative activities.
- To provide a scalable platform capable of supporting thousands of students.

- **Key Functionalities**

Module	Functionality
Student Management	Student registration, profile and academic records
Course Management	Course creation, enrollment and registration
Attendance	Attendance recording and monitoring
Assessment	Assignment, internal assessment and marks management
Examination	Exam scheduling and result processing
Faculty Management	Faculty profiles and academic activities
Performance Monitoring	Monitoring marks, attendance and progress
At-Risk Detection	Identification of academically weak students
Notification	Alerts for students, faculty and advisors
Reporting	Academic and institutional reports
Dashboard	Performance and administrative statistics
Administration	User, department and academic-year management

2. Repository Organization

- **Repository Structure**

The project repository should follow a modular structure so that each component has a clearly defined responsibility.

Recommended Repository Structure

- **University-Academic-Management/**
 - Main project repository containing all application source code, documentation, tests, and configuration files.
- **README.md**
 - Provides an overview of the project.
 - Explains the project objectives, features, installation, and usage.
- **requirements.txt**
 - Contains the Python dependencies required to run the project.
- **.gitignore**
 - Specifies files and folders that should not be uploaded to GitHub.
 - Helps keep unnecessary or sensitive files out of the repository.
- **app.py**
 - Entry point of the application.
 - Starts and configures the backend application.
- **frontend/**
 - Contains frontend static resources.
 - **css/** – Stylesheets for the application.
 - **js/** – JavaScript files for frontend functionality.
 - **images/** – Images and other visual resources.
- **tests/**
 - Contains test cases used to validate application functionality.
 - **test_student.py** – Tests student management.
 - **test_course.py** – Tests course functionality.
 - **test_attendance.py** – Tests attendance functionality.
 - **test_performance.py** – Tests academic performance.
- **docs/**
 - Contains project documentation.

- installation.md – Installation and setup instructions.
- user-guide.md – Instructions for using the application.

- **Folder Organization Evaluation**

The repository is organized into separate modules according to their functionality. Source code, templates, static resources, tests, and documentation are maintained separately.

This organization improves maintainability because developers can locate the required files easily. For example, attendance-related functionality can be maintained in the attendance module without affecting unrelated student or examination modules.

The separation of responsibilities also makes collaborative development easier because different developers can work on different modules independently.

- **Maintainability**

The repository structure supports maintainability through:

- Modular source code.
- Separation of application components.
- Dedicated testing directory.
- Centralized documentation.
- Consistent file naming.
- Separation of frontend and backend resources.
- Use of configuration files for dependencies.

3. Code Quality Review

- **Readability**

The source code should be written using meaningful variable names, function names, and class names.

For example:

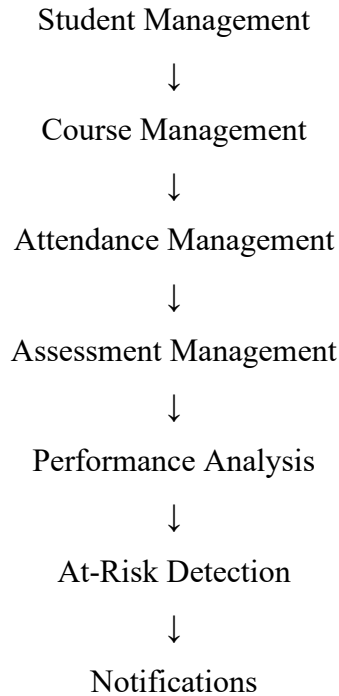
```
def calculate_attendance_percentage(attended_classes, total_classes):
    if total_classes == 0:
        return 0
    return (attended_classes / total_classes) * 100
```

The function name clearly describes its purpose, making the code easier to understand.

- **Modularity**

The application should be divided into smaller functions and modules rather than implementing the complete application in a single file.

For example:



This modular design allows individual components to be modified or tested independently.

- **Coding Standards**

The following coding practices should be followed:

- Use meaningful variable names.
- Use functions for repeated operations.
- Avoid unnecessary duplicate code.
- Maintain consistent indentation.
- Use comments where necessary.
- Use appropriate exception handling.
- Keep functions small and focused.
- Separate business logic from presentation logic.
- Follow a consistent naming convention.

- **Documentation in Code**

Important functions should contain comments or documentation explaining their purpose.

Example:

```
def identify_at_risk_student(attendance, marks):  
    """  
    Identifies students who may be academically at risk  
    based on attendance and academic performance.  
    """  
    return attendance < 75 or marks < 50
```

This improves code maintainability and helps new developers understand the implementation.

- **Strengths Identified**

The major strengths of the proposed implementation are:

- Modular design.
- Clear separation of academic modules.
- Reusable functions.
- Meaningful naming conventions.
- Centralized academic information.
- Automated academic monitoring.
- At-risk student identification.
- Report generation.
- Support for multiple user roles.
- Potential for future scalability.

- **Areas for Improvement**

The following improvements can be considered:

- Increase code documentation.
- Add more automated tests.
- Improve exception handling.
- Reduce duplicate code.
- Use configuration files instead of hard-coded values.
- Implement stronger input validation.
- Improve security mechanisms.
- Add logging for important system events.

- Follow a standard coding style consistently.

4. Version Control Evaluation

- **Git Repository**

Git is used to maintain different versions of the project source code. The repository provides a complete history of project development and allows developers to track changes.

The Git repository should contain:

- Source code.
- Documentation.
- Configuration files.
- Test cases.
- README file.
- Dependency information.

- **Commit History**

Commit history should contain meaningful messages describing the changes made.

Good Commit Messages

- Initial project setup
- Add student management module
- Implement course registration
- Add attendance monitoring
- Implement examination module
- Add student performance dashboard
- Implement at-risk student detection
- Add academic report generation
- Update project documentation
- Fix attendance calculation bug
- Meaningful commit messages make it easier to understand the development history.

- **Branching Strategy**

A suitable branching strategy is:

- main
 - feature/student-management
 - feature/course-registration
 - feature/attendance

feature/examination

feature/performance-analysis

The main branch contains the stable version of the project, while feature branches are used to develop individual functionalities.

- **Version Control Benefits**

Git supports collaborative software development by:

- Tracking code changes.
- Maintaining previous versions.
- Supporting multiple developers.
- Providing branching and merging.
- Preventing accidental loss of code.
- Supporting code review.
- Making debugging easier.
- Maintaining a development history.

- **Repository Maintenance**

The repository should be maintained by:

- Removing unnecessary files.
- Maintaining a proper .gitignore.
- Keeping documentation updated.
- Avoiding unnecessary large files.
- Using meaningful commit messages.
- Maintaining stable main branch code.
- Reviewing pull requests before merging.

5. Code Testing and Validation

- **Testing Objective**

Testing is performed to verify that the platform functions correctly and produces the expected results.

The testing process covers major modules including student management, course registration, attendance, examination, performance analysis, and at-risk student identification.

- **Test Cases**

Test ID	Test Case	Input	Expected Result	Status
TC01	Student registration	Valid student details	Student successfully registered	Pass
TC02	Student login	Valid credentials	Dashboard displayed	Pass
TC03	Course registration	Valid course selection	Course successfully registered	Pass
TC04	Attendance calculation	36/40 classes	Attendance = 90%	Pass
TC05	Low attendance detection	Attendance = 65%	Warning generated	Pass
TC06	Marks entry	Valid marks	Marks stored successfully	Pass
TC07	Result calculation	Valid assessment marks	Correct result generated	Pass
TC08	At-risk detection	Low marks + low attendance	Student identified as at risk	Pass
TC09	Notification	At-risk student detected	Faculty/advisor notification generated	Pass
TC10	Report generation	Academic data	Report generated successfully	Pass

- **Attendance Validation**

For example, if a student attends 36 out of 40 classes:

Attendance Percentage

$$= (36 / 40) \times 100$$

$$= 90\%$$

The system should correctly display the student's attendance as **90%**.

- **At-Risk Student Validation**

The system can use configurable academic policies.

Example:

Attendance < 75%

OR

Academic Performance < 50%

If either condition is satisfied, the student can be flagged for academic intervention.

Example:

Student: S101

Attendance: 68%

Average Marks: 45%

Status: Academically At Risk

The system can then notify the faculty member and academic advisor.

- **Testing Results**

The test results demonstrate that the core functionalities operate according to the defined requirements.

Testing validates:

- Correct student registration.
- Correct course enrollment.
- Accurate attendance calculation.
- Correct examination processing.
- Accurate performance calculation.
- Proper identification of at-risk students.
- Successful report generation.
- Correct notification functionality.

6. Repository Documentation

- **README Evaluation**

The README.md file should provide sufficient information for developers and users to understand and execute the project.

The README should contain:

- Project title.
- Stack
- Features.
- Run with Docker
- Project structure.
- Installation instructions.
- Testing instructions.

- **Installation Instructions**

The project documentation should clearly explain how to install and execute the application.

Example:

```
git clone <repository-url>
cd University-Academic-Management
pip install -r requirements.txt
python src/main.py
```

The actual commands should be replaced according to the technologies used in the implemented project.

- **Dependencies**

All external libraries required by the project should be documented.

For example:

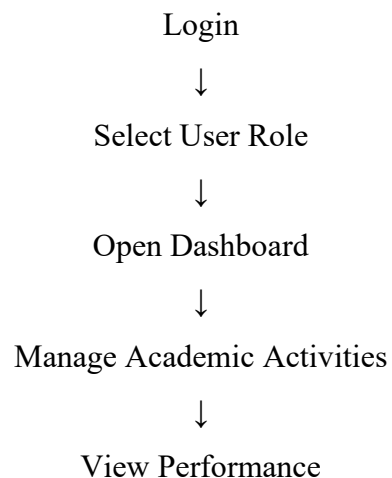
- Python
- Flask
- Pandas
- NumPy
- SQLite/MySQL
- Bootstrap
- JavaScript

The actual dependency list should match the technologies used in the project.

- **Usage Guide**

The usage guide should explain the major workflows.

Example:





Generate Reports

- **Documentation Strengths**

The documentation provides a foundation for understanding the project and can help new developers install and use the application.

- **Documentation Improvements**

The documentation can be improved by:

- Adding detailed installation steps.
- Including screenshots.
- Providing sample input data.
- Explaining each module.
- Adding troubleshooting instructions.
- Documenting API endpoints if applicable.
- Adding database schema documentation.
- Adding testing instructions.
- Maintaining a changelog.

7. Code Review Findings and Recommendations

- **Overall Findings**

The University Academic Management and Student Success Platform provides an integrated approach to university academic administration. The system combines student management, course registration, attendance, assessments, examinations, performance monitoring, notifications, and reporting.

The modular architecture improves maintainability and allows individual components to be enhanced independently.

The use of Git provides version tracking and supports collaborative software development.

Testing demonstrates that the major functionalities can be validated using defined test cases.

- **Code Quality Recommendations**

The following improvements are recommended:

1. Follow consistent coding standards.
2. Use meaningful variable and function names.
3. Add documentation strings to important functions.
4. Reduce duplicate code.

5. Improve exception handling.
6. Implement stronger input validation.
7. Use modular architecture.
8. Add automated unit tests.
9. Implement logging.
10. Conduct periodic code reviews.

- **Repository Recommendations**

The repository should:

1. Maintain a clean folder structure.
2. Use meaningful commit messages.
3. Follow a feature-branch workflow.
4. Maintain a stable main branch.
5. Use .gitignore.
6. Remove unnecessary files.
7. Maintain project documentation.
8. Use pull requests for major changes.
9. Review code before merging.
10. Tag stable releases.

- **Testing Recommendations**

Testing can be improved by:

- Increasing test-case coverage.
- Adding unit testing.
- Adding integration testing.
- Testing invalid inputs.
- Testing boundary conditions.
- Testing different user roles.
- Testing concurrent users.
- Testing performance with large datasets.
- Testing system recovery.
- Automating tests through CI/CD.

- **Security Recommendations**

Since the platform handles sensitive academic information, security should be considered a major requirement.

Recommended measures include:

- Secure user authentication.
- Role-based access control.
- Password hashing.
- Input validation.
- Protection against SQL injection.
- Secure session management.
- HTTPS communication.
- Regular security testing.
- Audit logging.
- Secure backup and recovery mechanisms.

- **Scalability Recommendations**

The platform should be capable of supporting thousands of students and multiple departments.

Future improvements include:

- Database optimization.
- Database indexing.
- Caching.
- Load balancing.
- Cloud deployment.
- API-based architecture.
- Horizontal scaling.
- Asynchronous processing.
- Monitoring and performance analytics.

- **Future Enhancements**

Future versions can include:

- Mobile application support.
- AI-assisted student performance prediction.
- Advanced academic analytics.
- Personalized student recommendations.

- Automated academic advising.
- Parent/guardian communication.
- Online learning integration.
- Library management integration.
- Placement management integration.
- Real-time institutional dashboards.

8. Overall Evaluation

The **University Academic Management and Student Success Platform** addresses the major challenges faced by higher education institutions in managing large-scale academic operations.

The centralized platform improves academic administration by integrating student records, course registration, attendance, assessments, examinations, faculty activities, notifications, and reporting.

The at-risk student identification feature enables early academic intervention by identifying students who may require additional academic support.

The repository-based development approach supports maintainability, collaboration, and version control. Proper documentation, testing, modular architecture, and Git-based development further improve the reliability and maintainability of the system.

Overall, the proposed platform has the potential to improve student success, faculty productivity, academic transparency, institutional efficiency, and evidence-based academic decision-making.

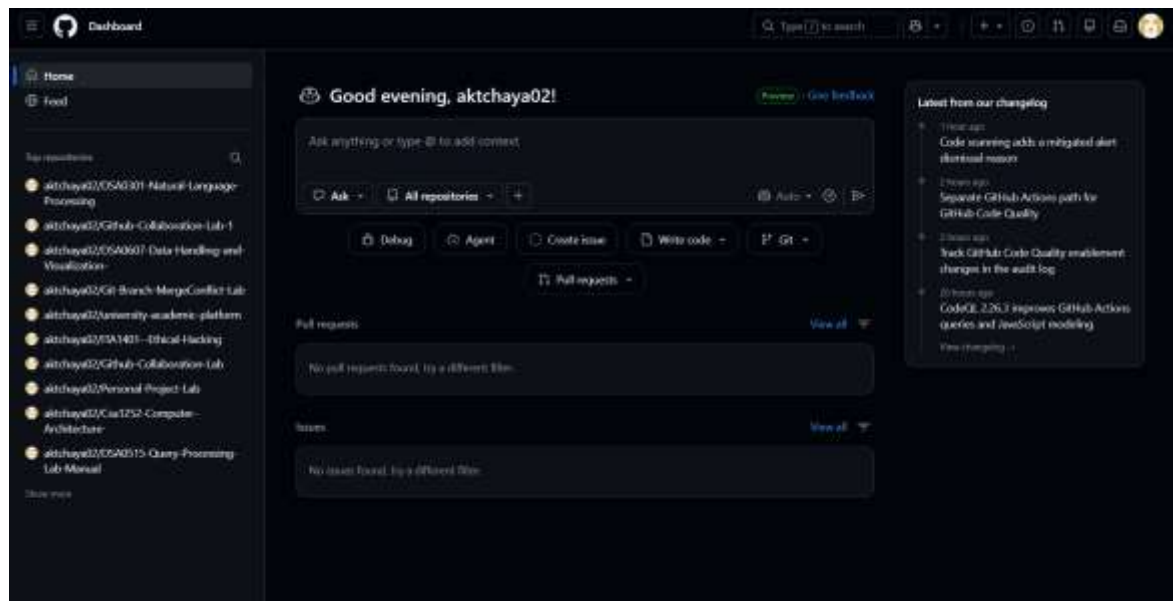
9. Expected Outcomes

The proposed platform is expected to achieve the following outcomes:

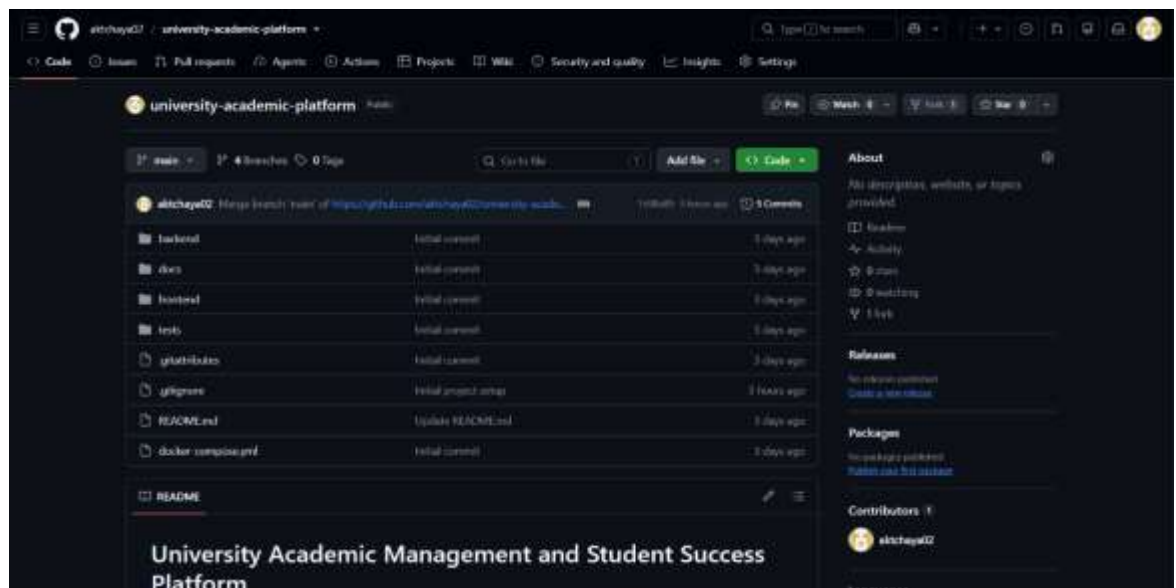
- Improve academic administration.
- Increase student success rates.
- Improve faculty productivity.
- Reduce manual administrative work.
- Improve academic transparency.
- Support timely academic interventions.
- Enhance institutional efficiency.
- Improve communication among stakeholders.
- Support evidence-based academic planning.
- Enable scalable university management.

Screenshots:

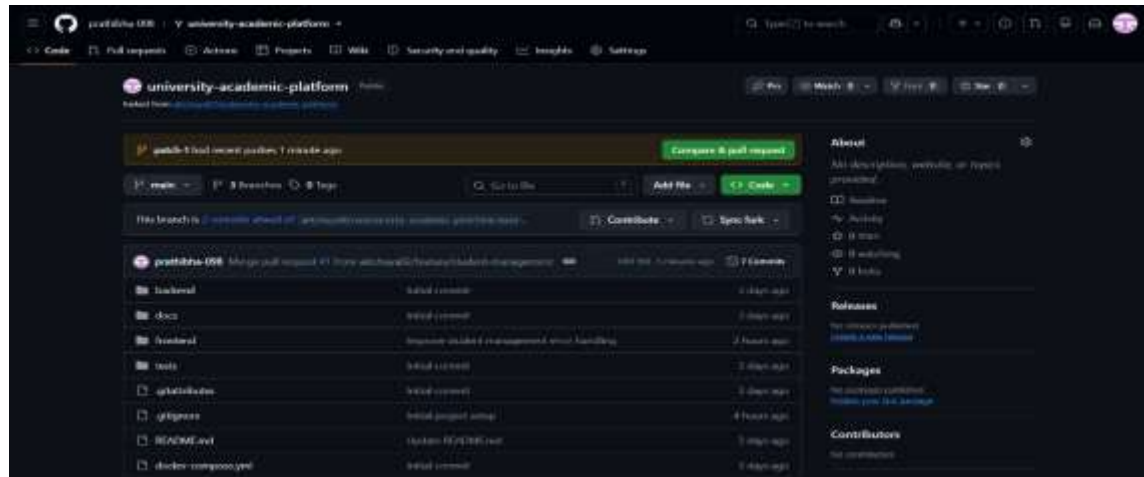
- Github Main Page



- Project repository



- Fork your project into another github account



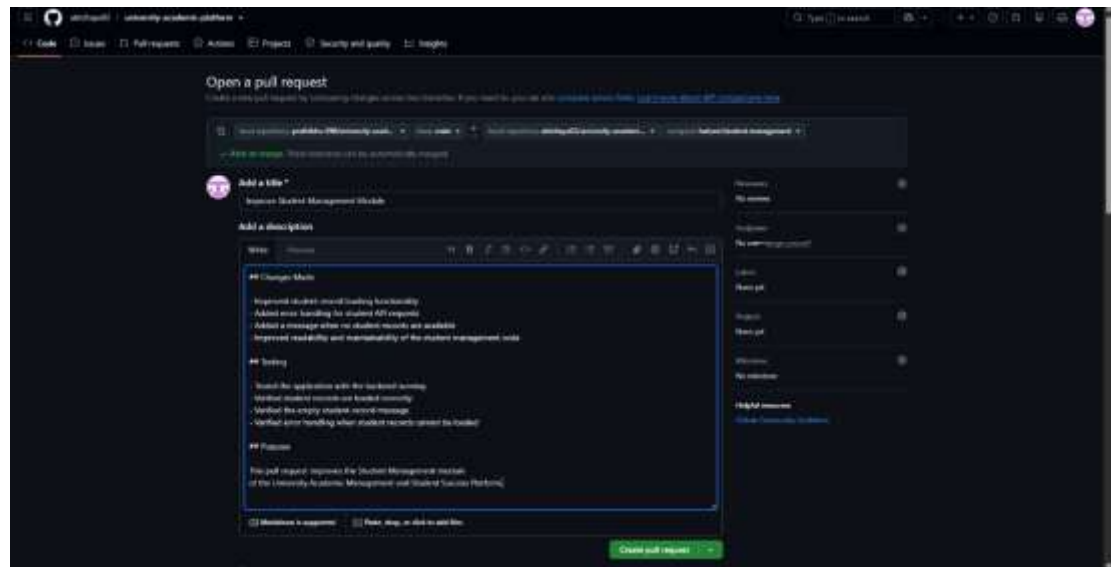
- Modify the code

```

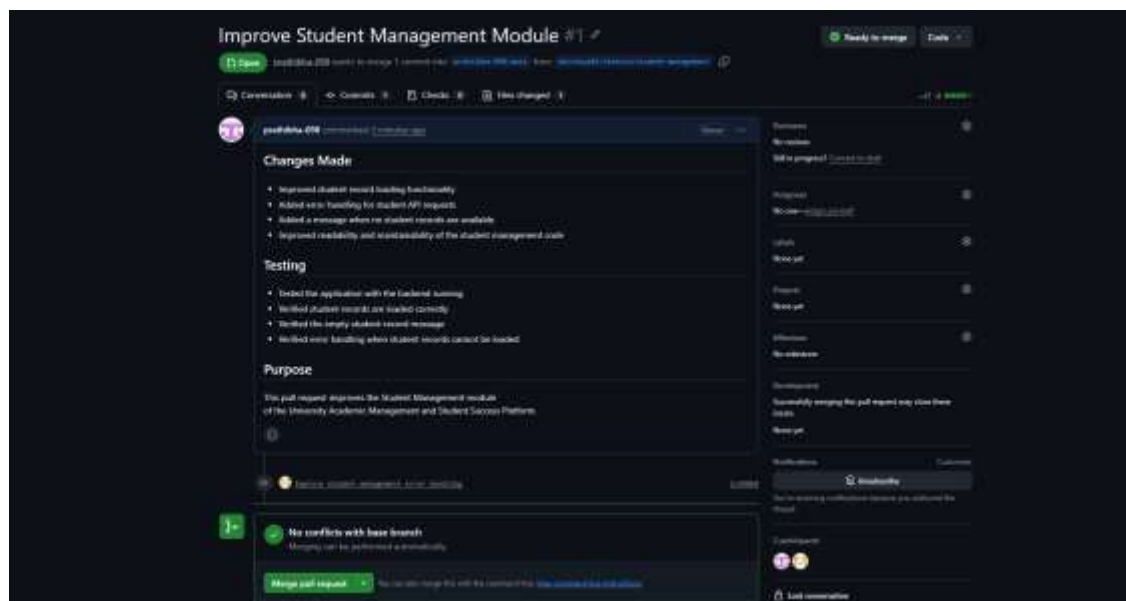
32   async function loadStudents() {
33       try {
34           const r = await fetch(`${API}/students`);
35
36           if (!r.ok) {
37               throw new Error("Unable to load student records");
38           }
39
40           const students = await r.json();
41           const table = document.getElementById("studentTable");
42
43           if (!students.length) {
44               table.innerHTML = `
45               <tr>
46                   <td colspan="7" style="text-align:center;">
47                       No student records found.
48                   </td>
49               </tr>
50           `;
51           return;
52       }

```

- Create Pull request with title and description



- Merge the Pull request



- After merging



